

OWASP Juice Shop

Web Security Assignment Submission

Student	Siva Sai Deepank Manoj
Email	deeps45@tamu.edu
GitHub (public)	https://github.com/deeps45/juice-guard
Demo account	demo@juice.shop / JuiceShop1!

Part 1 — Secure Feature Design

I looked through OWASP Juice Shop with a focus on registration and login. Three issues stood out, and each one shaped how I would design a safer signup flow.

1. SQL injection on login. If the login query is built by joining strings, a crafted email can change the query and skip password checks. I would only look up users with parameterized queries (or an ORM), and I would validate the email format first. That way user input stays data, not code.

2. Cross-site scripting (XSS). Fields like search or product reviews can store or reflect HTML and JavaScript. If that content is rendered raw, it can run in another user's browser. I would encode output, prefer `textContent` over `innerHTML`, reject angle brackets on input, and add a `Content-Security-Policy` header so scripts cannot run inline.

3. Authentication bypass / weak sessions. Weak or predictable tokens make it easier to impersonate another account. I would use signed tokens with short expiry, check authorization on the server for every sensitive route, rate-limit login attempts, and always return a generic "Invalid email or password" message so attackers cannot tell which field was wrong.

Secure password handling

Passwords should never be stored in plaintext. On registration I hash once with `bcrypt` (cost factor 12). On login I compare with `bcrypt.compare`. Even if the database leaks, attackers only get slow-to-crack hashes.

```
const passwordHash = await bcrypt.hash(password, 12);
// store passwordHash only – never the plaintext password

const ok = await bcrypt.compare(submittedPassword, user.passwordHash);
if (!ok) return res.status(401).json({ error: 'Invalid email or password.' });
```

Part 2 — Front-End Login Form

I built a simple Juice Shop–style login page in HTML, CSS, and JavaScript, with a small Express server behind it. The form has email and password fields, a submit button, and an on-page status message.

Client-side validation (`public/app.js`) runs before any network call. Empty submissions are blocked. The email must contain `@`. The password must be at least 8 characters. If a check fails, the field

is highlighted and a short error is shown.

Server-side validation (`server.js`) repeats the checks more strictly: email shape, password length, and rejection of unsafe characters. Users are stored in memory and looked up by email — there is no string-built SQL. Passwords are verified with `bcryptjs`. Responses use safe text rendering (`textContent`), and login is rate-limited.

How to run

```
git clone https://github.com/deeps45/juice-guard.git
cd juice-guard
npm install
npm start
# open http://127.0.0.1:3847
# demo login: demo@juice.shop / JuiceShop1!
```

Public GitHub repository: <https://github.com/deeps45/juice-guard>

The README explains what the project does and how to run it.

The screenshot shows the 'Sign in securely' login form. It has two input fields: 'Email' with the value 'you@example.com' and 'Password' which is empty. Below the password field is a hint 'At least 8 characters'. A yellow 'Log in' button is at the bottom. A footer line reads 'Demo account: [demo@juice.shop](#) / [JuiceShop1!](#)'.

Figure 1. Login form (email + password).

The screenshot shows the successful login state. The 'Email' field now contains 'demo@juice.shop' and the 'Password' field is filled with eight asterisks. The 'Log in' button is still present. A green message 'Welcome back, demo@juice.shop!' is displayed above the footer line 'Demo account: [demo@juice.shop](#) / [JuiceShop1!](#)'.

Figure 2. Successful login after both checks.

The screenshot shows the login form with validation errors. The 'Email' field has a red border and a red error message 'Email cannot be empty.' below it. The 'Password' field also has a red border and a red error message 'Password cannot be empty.' below it. A red message 'Fix the highlighted fields before submitting.' is at the bottom. The footer line remains 'Demo account: [demo@juice.shop](#) / [JuiceShop1!](#)'.

Figure 3. Empty fields blocked on the client.

The screenshot shows the login form with a specific validation error. The 'Email' field is correct, but the 'Password' field has a red border and a red error message 'Password must be at least 8 characters.' below it. A red message 'Fix the highlighted fields before submitting.' is at the bottom. The footer line remains 'Demo account: [demo@juice.shop](#) / [JuiceShop1!](#)'.

Figure 4. Password must be at least 8 characters.

Part 3 — Trying to Break My Own Form

After building the form, I tried the same kinds of attacks Juice Shop is known for — XSS and SQL injection — against my own login page.

Attempt 1: Cross-site scripting (XSS)

Steps. I opened the login page, entered `xss@test.com` as the email, and put `<script>alert(1)</script>` in the password field. Then I clicked Log in and watched for a JavaScript alert or any script execution in the page.

Result. The attack did not work. No alert appeared. The server rejected the password because it contained unsafe characters (`<` / `>`). The UI also writes status messages with `textContent`, so even a reflected string would not be treated as HTML.

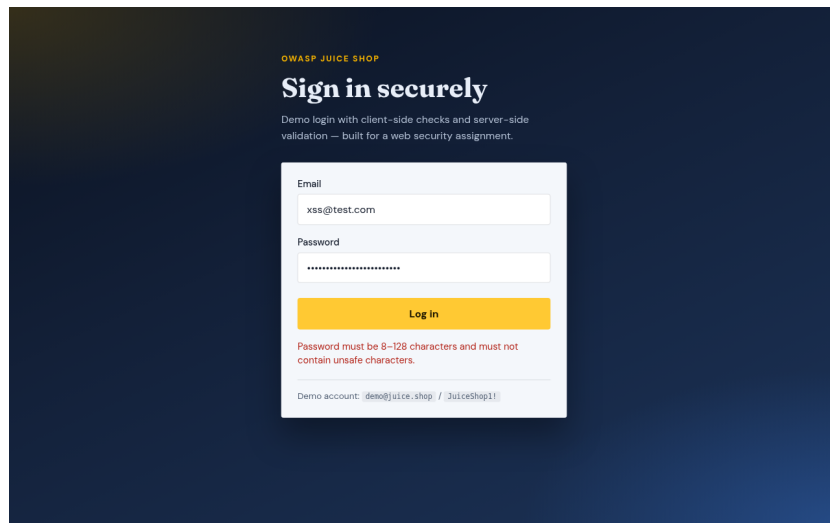


Figure 5. XSS attempt blocked — no script ran in the browser.

Attempt 2: SQL injection

Steps. I reloaded the page and entered `admin@test.com' OR '1'='1` as the email and `password1` as the password, then submitted the form. I checked whether login succeeded or any credentials were exposed.

Result. The attack did not work. The app treated the email as invalid and never built a SQL string from user input (accounts live in an in-memory map keyed by email). There was no authentication bypass and no data leak.

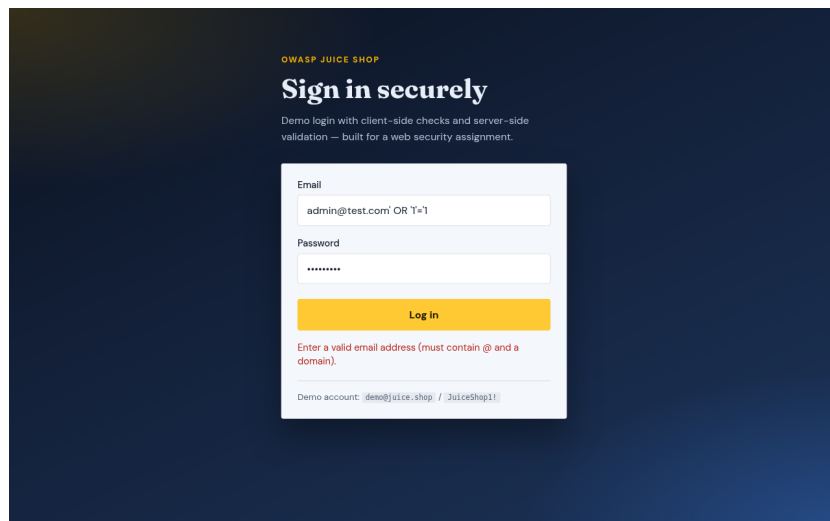


Figure 6. SQL injection-style email rejected by validation.

One fix I applied

Even though both attacks failed, I added a Content-Security-Policy header (default-src 'self'; script-src 'self', with limited exceptions for fonts). If someone later uses innerHTML by mistake, the browser still blocks inline script. That is useful defense in depth on top of input checks and bcrypt hashing.

Overall takeaway: client checks help with usability, but server validation, safe DOM updates, and never concatenating SQL are what actually stop XSS and injection. Not being able to break my own form was a good sign.

End of submission · Source and README: <https://github.com/deeps45/juice-guard>